RAJALAKSHMI
ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY Chennai

BONAFIDE CERTIFICATE

NAME .....Brinda.....C.V.....

ACADEMIC YEAR .....3..... SEMESTER .....V.....BRANCH.....CSD.....

UNIVERSITY REGISTER No. 2116241701011

Certified that this is the bonafide record of work done by the above student in the

.....Graphics & Multimedia..... Laboratory during the year 2026 - 2027

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on ....03.11.2026.....

Internal Examiner

External Examiner

INDEX

NAME: Brinda STD.: SEC.: ROLL NO.: SUB.:

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	16/07/26	Implement bresenham line, midpoint and cable algorithm		
2.	16/07/26	Implement the scan conversation of a polygon and use flood fill.		
3.	24/07/26	Write a program that performs translation Scaling and Rotation on 2D.		
4.	31/07/26	Write a program that clips polygon to a specific window		
5.	07/08/26	Write a program that allows user to perform 3D transformation		
6.	14/08/26	Create and render 3D objects and apply basic color and shading.		
7.	21/08/26	Create a media app that integrates img, audio, video.		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
8.	11/09/26	Create a program that captures video/Audio from a Webcam.		
9.	25/09/2026	Create an app that allows users to send and receive multimedia text, img and audio		
10.	09/10/26	Creata a Simple 3D model using Blender		

Completed
14/10/26

17/7/2026

Experiment - 1

Bresenham's line Drawing Algorithm

Aim:

To write and execute a python program to implement Bresenham's line Drawing Algorithm for drawing a straight line where the slop $|m|<1$.

Code:

```
def bresenham(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    p = 2 * abs(dy) - abs(dx)
    x = x1
    y = y1
    y_step = 1 if dy >= 0 else -1
    print("Points on the line:")
    print(f"({x}, {y})")
    while x < x2:
        x += 1
        if p < 0:
            p = p + 2 * abs(dy)
        else:
            y += y_step
            p = p + 2 * abs(dy) - 2 * abs(dx)
        print(f"({x}, {y})")
x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
x2 = int(input("Enter x2: "))
y2 = int(input("Enter y2: "))
bresenham(x1, y1, x2, y2)
```

Result:

Thus, the python program to implement Bresenham line drawing Algo for drawing a line with $|m|<1$ was successfully executed.

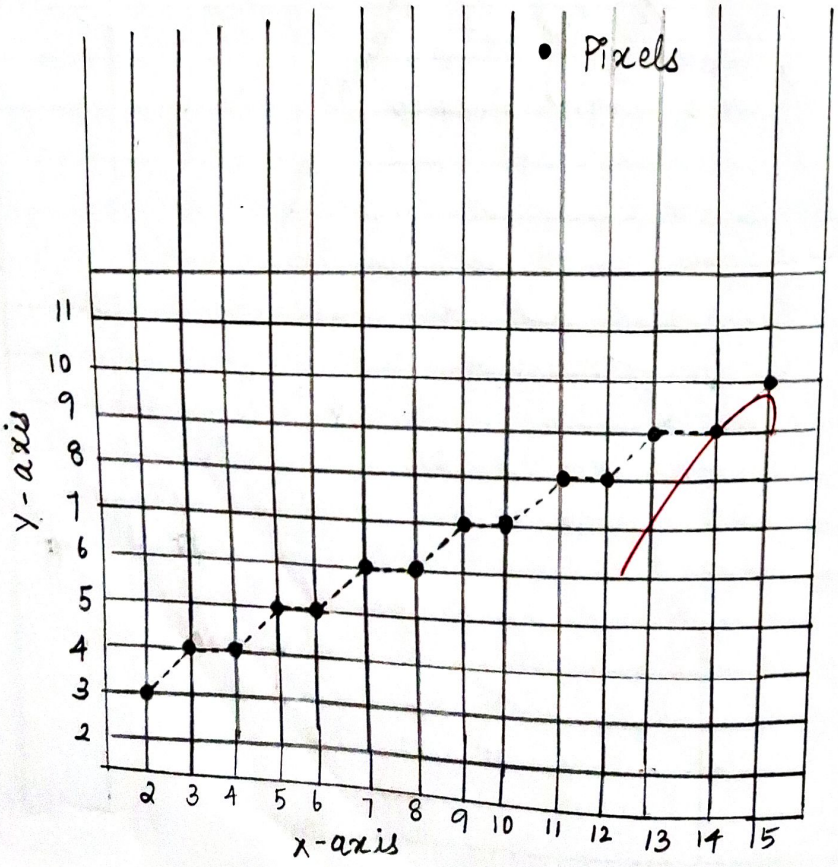
Sample Input:

Enter x1: 2
Enter y1: 3
Enter x2: 10
Enter y2: 6

Sample Output:

Points on the line:

(2,3)
(3,3)
(4,4)
(5,4)
(6,4)
(7,5)
(8,5)
(9,6)
(10,6)



24/07/26

Bresenham's Square Drawing Algorithm

Aim:

To draw a square using Bresenham's line drawing Algorithm in python.

code:

```
import matplotlib.pyplot as plt.
# Square coordinates
x = [2,8,8,2,2]
y = [2,2,8,8,2]
# Draw Square
plt.figure(figsize=(6,6))
plt.plot(x, y, color='blue', linewidth=2)
plt.scatter(x,y, color='red', s=50)
plt.title("Square")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.axis("equal")
plt.show()
```

29/7/26

Result:

The square was successfully drawn by connecting four vertices using the Matplotlib library in python.

Sample Input:

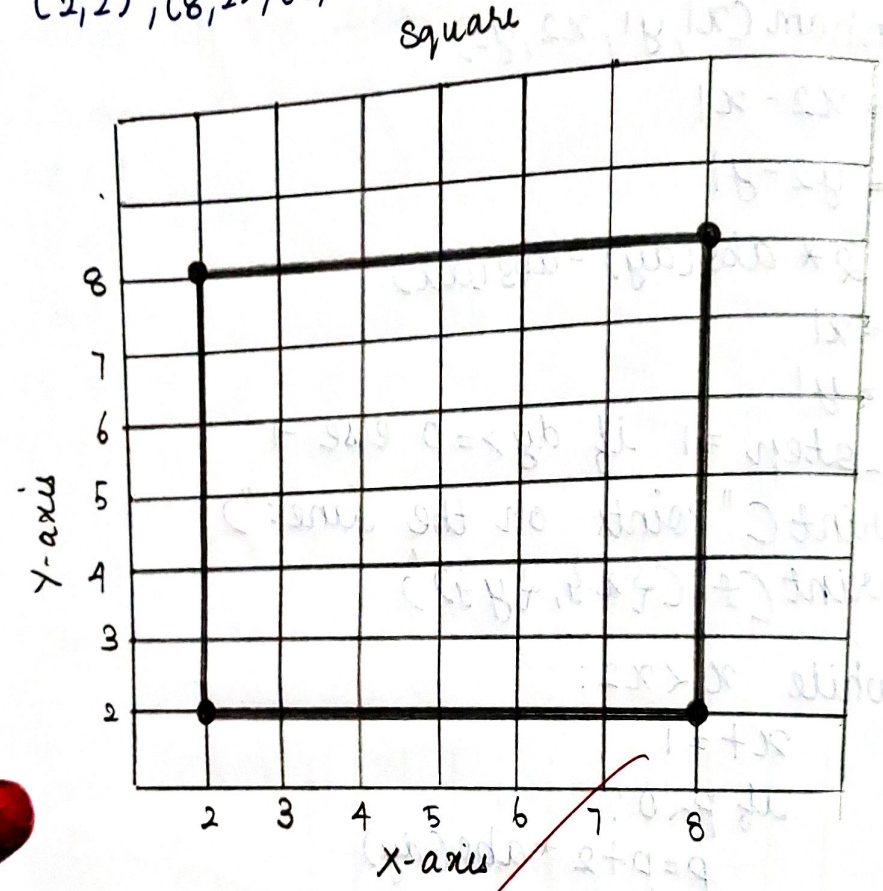
x = [2, 8, 8, 2, 2]

y = [2, 2, 8, 8, 2]

Sample Output:

A square is displayed with vertices:

(2,2), (8,2), (8,8), (2,8)



Draw a circle Using Bresenham's Line Algorithm

Aim:

To draw a circle by connecting multiple points with line segments using python and the Matplotlib library.

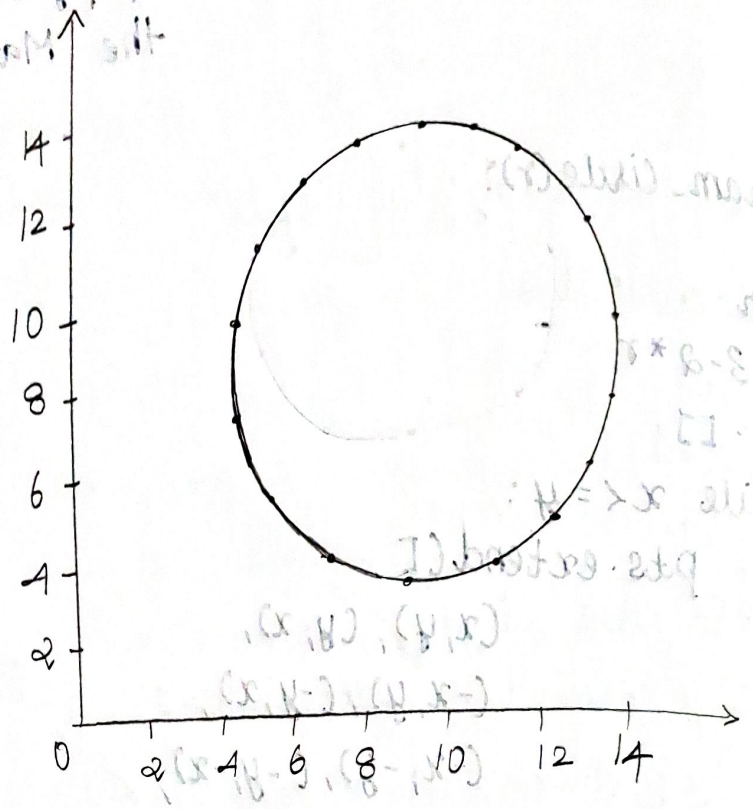
code:

```
imports
def bresenham_circle(r):
    x=0
    y=r
    d=3-2*r
    pts=[]
    while x<=y:
        pts.extend([
            (x,y),(y,x),
            (-x,y),(-y,x),
            (-x,-y),(-y,-x),
            (x,-y),(y,-x)
        ])
        if d<0:
            d=d+4*x+6
        else:
            d=d+4*(x-y)+10
            y-=1
        x+=1
    return pts
radius=int(input("Enter radius:"))
points=list(set(bresenham_circle(radius)))
points.sort(key=lambda p: np.arctan2(p[1],p[0]))
x=np.array([p[0] for p in points])
y=np.array([p[1] for p in points])
x=np.append(x,x[0])
y=np.append(y,y[0])
```

```
tck, u = splprep([x, y], s=0, per=True)
u_new = np.linspace(0,1.0, 1000)
x_new, y_new = splev(u_new, tck)
plt.figure(figsize=(6,6))
plt.plot(x_new, y_new, linewidth=2)
plt.axis('equal')
plt.grid(True)
plt.title("Smooth Circle using Bresenham
 Points and Spline Interpolation").
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.show()
```

Result:

A circle was successfully drawn using Bresenham's circle algorithm with the decision variable, utilizing 8-way symmetry for efficient circle generation.


14
12
10
8
6
4
2
0
2
4
6
8
10
12
14

Draw a Rectangle using Bresenham's Algorithm

Aim:

To draw a rectangle using Bresenham's Algorithm.

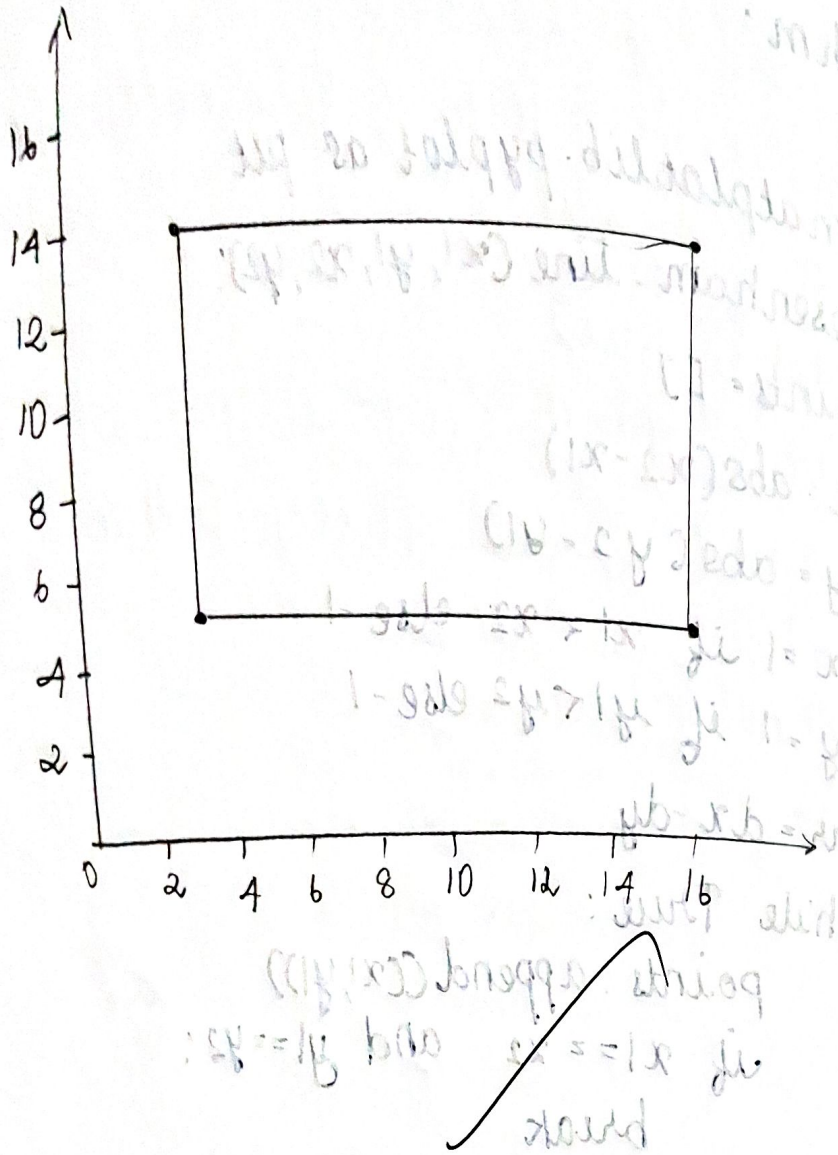
Code:

```
import matplotlib.pyplot as plt
def bresenham_line(x1, y1, x2, y2):
    points = []
    dx = abs(x2-x1)
    dy = abs(y2-y1)
    sx = 1 if x1 < x2 else -1
    sy = 1 if y1 < y2 else -1
    err = dx-dy
    while True:
        points.append((x1,y1))
        if x1 == x2 and y1 == y2:
            break
        e2 = 2 * err
        if e2 > -dy:
            err -= dy
            x1 += sx
        if e2 < dx:
            err += dx
            y1 += sy
    return points
edges = [(2,2,10,2),
         (10,2,10,7),
         (10,7,2,7),
         (2,7,2,2)]
x_points = []
y_points = []
```

```
for x1, y1, x2, y2 in edges:
    line = bresenham_line(x1, y1, x2, y2)
    for x, y in line:
        x_points.append(x)
        y_points.append(y)
plt.plot(x_points, y_points, linewidth=2)
plt.title("Rectangle using Bresenham
          line Algorithm")
plt.grid(True)
plt.axis("equal")
plt.show()
```

Result:

Hence the Rectangle is drawn using Bresenham's Algorithm.


16
14
12
10
8
6
4
2
0
2
4
6
8
10
12
14
16

DRAW THE OLYMPHIC LOGO USING ELLIPSE (MID POINT ALGO)

Aim:

To draw the olymphic logo using ellipse and mid-point algorithm.

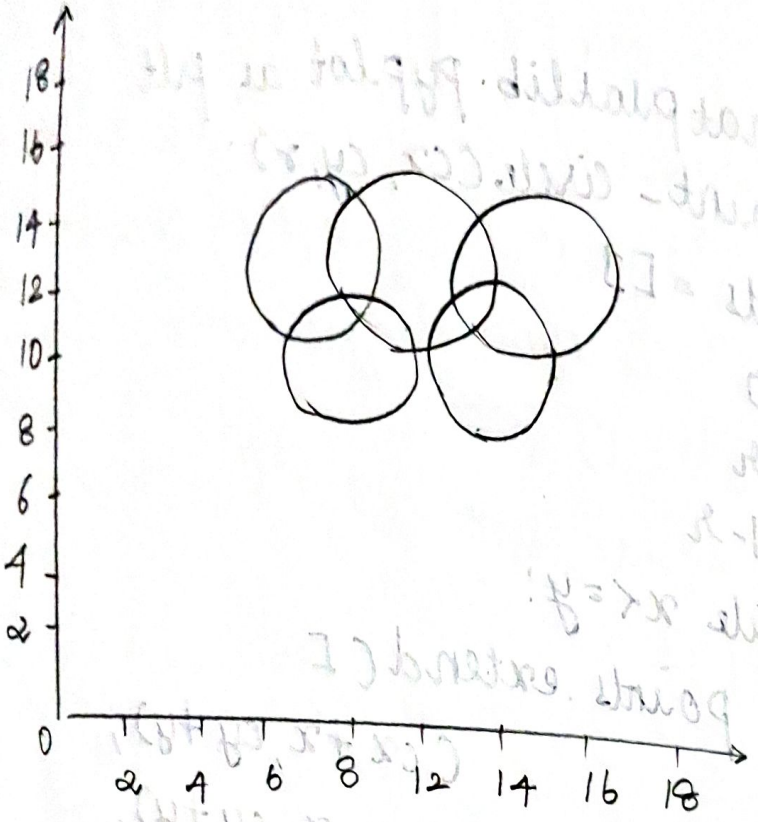
code:

```
import matplotlib.pyplot as plt
def midpoint_circle(cx, cy, r):
    points = []
    x = 0
    y = r
    p = 1-r
    while x<=y:
        points.extend([
                (cx+x, cy+y),
                (cx-x, cy+y),
                (cx+x, cy-y),
                (cx-x, cy-y),
                (cx+y, cy+x),
                (cx-y, cy+x),
                (cx+y, cy-x),
                (cx-y, cy-x)
        ])
        if p<0:
            p = p+2*x+3
        else:
            p = p+2*(x-y)+5
            y-=1
        x+=1
    return points
def draw_circle(cx, cy, r):
    points = midpoint_circle(cx, cy, r)
```

```
    points.sort(key=lambda p: __import__
        ('math').atan2(p[1]-cy, p[0]-cx))
    x = [p[0] for p in points]
    y = [p[1] for p in points]
    plt.plot(x, y, linewidth=3)
r=50
draw_circle(0, 60, r)
draw_circle(110, 60, r)
draw_circle(220, 60, r)
draw_circle(55, 0, r)
draw_circle(165, 0, r)
plt.axis("equal")
plt.axis("off")
plt.show()
```

Result:-

The Olympic logo using ellipse is successfully.


18
16
14
12
10
8
6
4
2
0
2
4
6
8
12
14
16
18

~~SHALBY RACE~~

DRAW ~~BUTTERFLY~~ BUTTERFLY USING MIDPOINT ALGORITHM.

Aim:

To draw a butterfly using midpoint Algorithm.

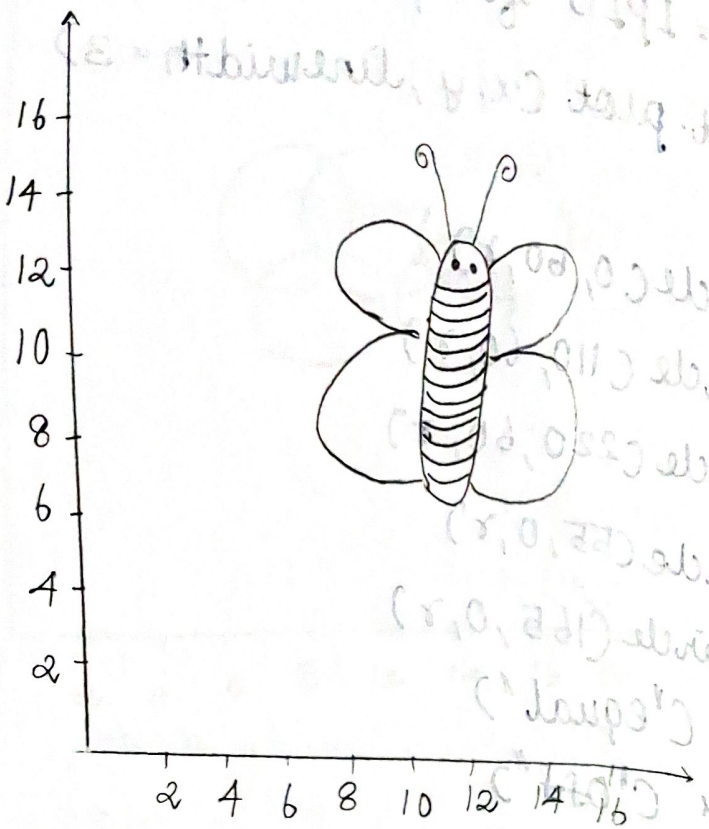
Code:

```
import numpy as np, matplotlib as plt.
def get-circle (xc, yc, r):
while x<=y:
    for dx, dy in
        pts.append((xc+dx, yc+dy))
    P, y = (P+2*x+3*y)
    x+=1
return pts
def plt-fill (pts, xc, yc, color)
    pts = sorted(set(pts), key = lambda
            P: np.artan2(P[1]-yc,
                         P[0]-xc))
    plt.figure(figsize=(6,6))
    plot-fill(get-ellipse(2,6,4,3)'hotpink')
    plot-fill(get-circle(5,6,1)'black')
    plot-fill(get-circle(5,9,4)'black')
    plt-axis('equal')
    plt.show()
    plt.grid(True)
```

Result:

Hence the butterfly is drawn Successfully

Output:-



DRAW SMILEY FACE USING MIDPOINT CIRCLE ALGORITHM.

Aim:-

To draw a smiley face using mid point circle Algorithm.

Code:-

```
import matplotlib.pyplot as plt
import numpy as np
#mid point circle Algorithm.
def mid_point_circle(xc, yc, r):
    points = []
    x = 0
    y = r
    p = 1-r
    while x<=y:
        points.extend([
            (xc+x, yc+y),
            (xc-x, yc+y),
            (xc+x, yc-y),
            (xc-x, yc-y),
            (xc+y, yc+x),
            (xc-y, yc+x),
            (xc+y, yc-x),
            (xc-y, yc-x)
        ])
        if p<0:
            p+=2*x+3
        else:
            p+=2*(x-y)+5
            y-=1
        x+=1
    return points
```

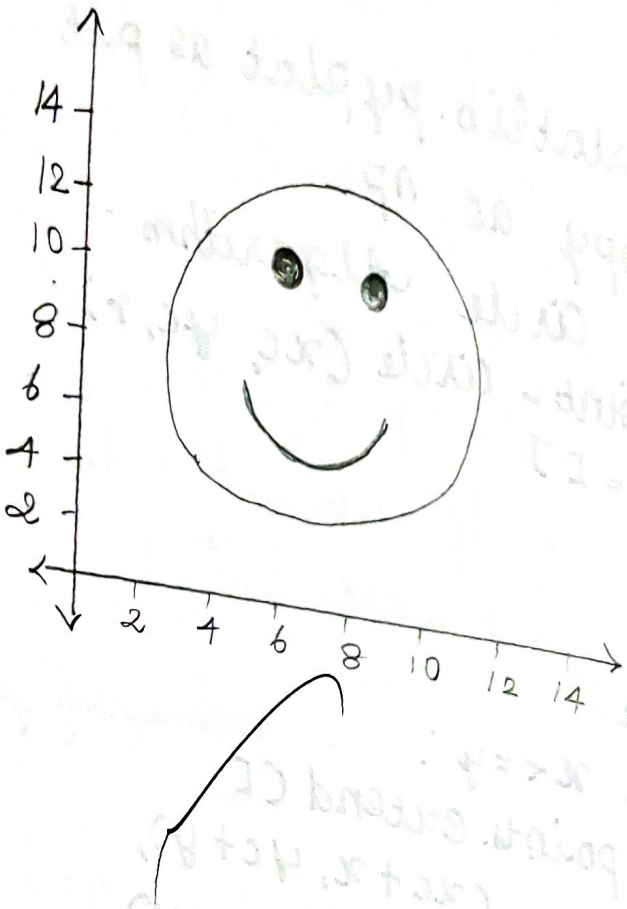
```
def filled_circle(xc, yc, r, color):
    y = np.arange(yc-r, yc+r, 0.01)
    for yy in y:
        half = np.sqrt(r*r-(yy-yc)**2)
        plt.plot(
            [xc-half, xc+half],
            [yy, yy],
            color=color
        )
    t = np.linspace(0, 2*np.pi, 300)
    x = xc + r*np.cos(t)
    y = yc + r*np.sin(t)
    plt.plot(x, y, color="black", linewidth=5)

filled_circle(0,0,5,"yellow")
filled_circle(0-1.8, 1.5, 0.5, "black")
filled_circle(1.8, 1.5, 0.5, "black")
t = np.linspace(np.pi+0.2, 2*np.pi-0.2, 150)
x = 2.5*np.cos(t)
y = 1.5*np.sin(t)-0.5
plt.show()
```

Result:-

To draw smiley face using midpoint algorithm is drawn successfully.

Output:



FLOOD FILL ALGORITHM.

Aim:-

To draw a polygon using flood fill Algorithm.

Code:-

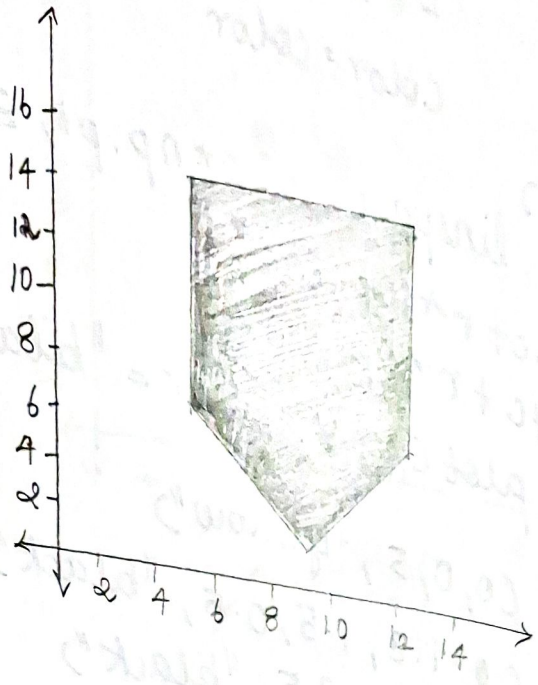
```
import matplotlib.pyplot as plt
import numpy as plt.
width = 300
height = 300
image = np.ones (cheight, width, 3), dtype=np.
                uint8) * 255
polygon = [(150, 50), (250, 130),
           (210, 240), (90, 240),
           (50, 130)]
def draw_line (x1, y1, x2, y2):
    points = max (abs (x2-x1), abs (y2-y1))
    for i in range (points+1)
        x = int (x1 + (x2-x1) * i/points)
        y = int (y1 + (y2-y1) * i/points)
        image [y, x] = [0, 0, 0]
for i in range (len(polygon)):
    x1, y1 = polygon [i]
    x2, y2 = polygon [(i+1) %. len(polygon)]
    draw_line (x1, y1, x2, y2)
def flood_fill(x, y, old_color, new_color):
    stack = [(x, y)]
    while stack:
        x, y = stack.pop()
        if x<0 or x>=width or y<0 or y>= heigh
            continue
```

```
    if not np.array_equal(image[y, x], old_
                                color):
        continue
    image[y, x] = new_color
    stack.append((x+1, y))
    stack.append((x-1, y))
    stack.append((x, y+1))
    stack.append((x, y-1))
flood_fill(
    150,
    150,
    np.array([255, 255, 255], dtype=np.uint8)
    ,np.array([0, 180, 0], dtype=np.uint8)
)
plt.imshow(image)
plt.title('Flood Fill Algorithm')
plt.axis('off')
plt.show()
```

Result:-

To draw the polygon using flood fill Algorithm is successfully drawn.

Output:-



Basic Transformations of Triangle.

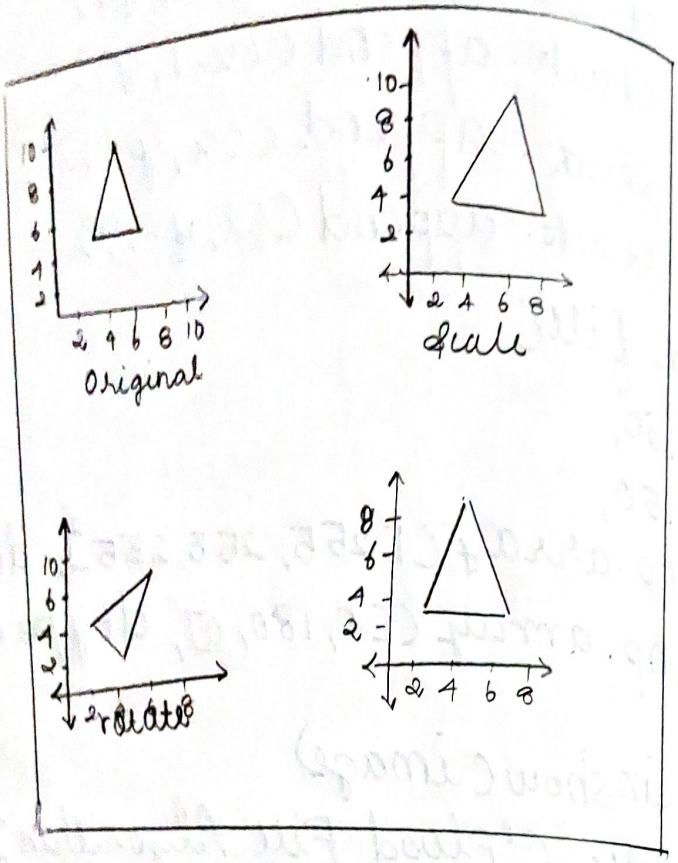
Aim:-

To perform basic transformations of triangle

code:-

```
import numpy as np
import matplotlib.pyplot as plt.
#Triangle
p = np.array([[2,2,1], [6,2,1], [4,5,1], [2,2,1]])
T=np.array # Translation
T = np.array([[1,0,3], [0,1,2], [0,0,1]])
translated = (T @ p.T).T
#Scaling
S = np.array([[1.5,0,0], [0,1.5,0], [0,0,1]])
scaled = (S@p.T).T
a = np.radians(45)
R=np.array([[np.cos(a), -np.sin(a), 0],
            [np.sin(a), np.cos(a), 0],
            [0,0,1]])
rotated = (R @ p.T).T
#Plot
fig, ax = plt.subplots(2, 2)
ax[0,0].plot(p[:,0], p[:,1])
ax[0,0].set_title("original")
ax[0,1].plot(translated[:,0], translated[:,1])
ax[0,1].set_title("Translation")
ax[1,0].plot(scaled[:,0], scaled[:,1])
ax[1,0].set_title("Scaling")
ax[1,1].plot(rotated[:,0], rotated[:,1])
ax[1,1].set_title('Rotation')
```

Output:-



```
for a in ax.flat:
    a.grid()
    a.set_aspect('equal')
plt.show()
```

Result:

Thus the Basic transformation for triangle is implemented successfully.

Transformation of polynomial Rectangle.

Aim:

To write a code to perform transformation of polynomial rectangle

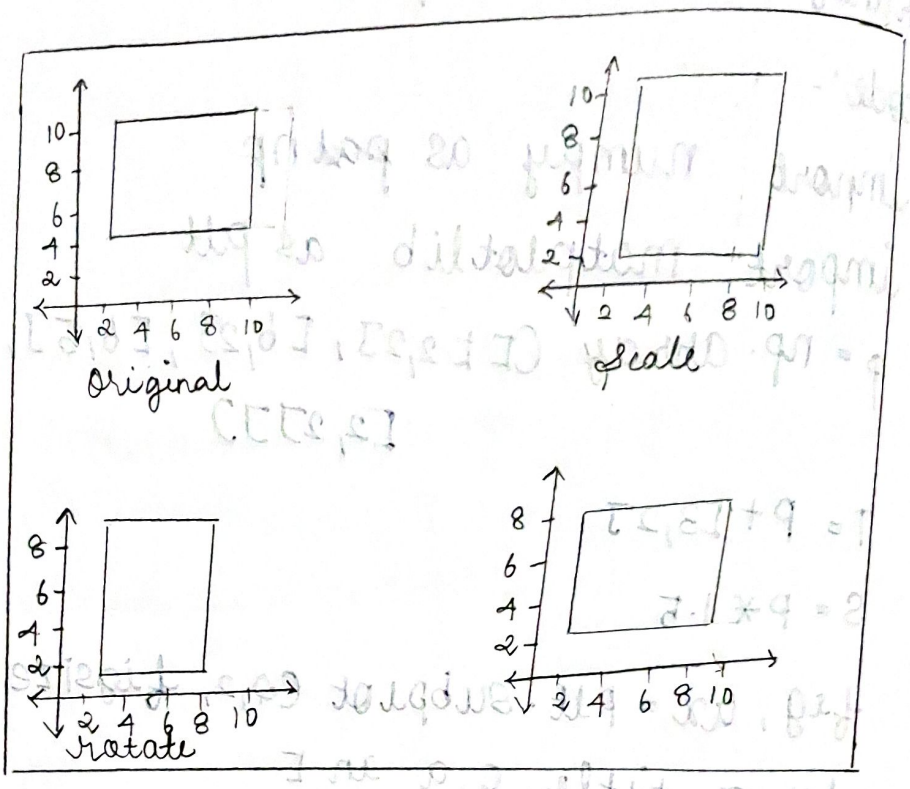
code:-

```
import numpy as np
import matplotlib as plt
p = np.array ([[2,2], [6,2], [6,5], [2,5],
               [2,2]])
T = P + [3,2]
S = P * 1.5
fig, ax = plt.subplot (2,2, figsize = (8,8))
for x, title, c, a in [
        (p, "Original", 'b', ax[0,0]),
        (T, "Translation", 'r', ax[0,1]),
        (S, "Scaling", 'g', ax[1,0]),
        (R, "Rotation", 'm', ax[1,1])]
a1.plot (x[:,0], x[:,1], c, linewidth = 2)
a1.fill (x[:,0], x[:,1], alpha = 2)
a1.set_title (title)
a1.set_aspect ('equal')
plt.tight_layout ()
plt.show ()
```

Result:

Thus the code is implimented and Output is obtained Successfully.

Output:-


10
8
6
4
2
2 4 6 8 10
Original
10
8
6
4
2
2 4 6 8 10
Scale
8
6
4
2
2 4 6 8 10
rotate
8
6
4
2
2 4 6 8 10

Ex:- 8 Polygon clipping.

Aim:-

To write a code to perform polygon clipping

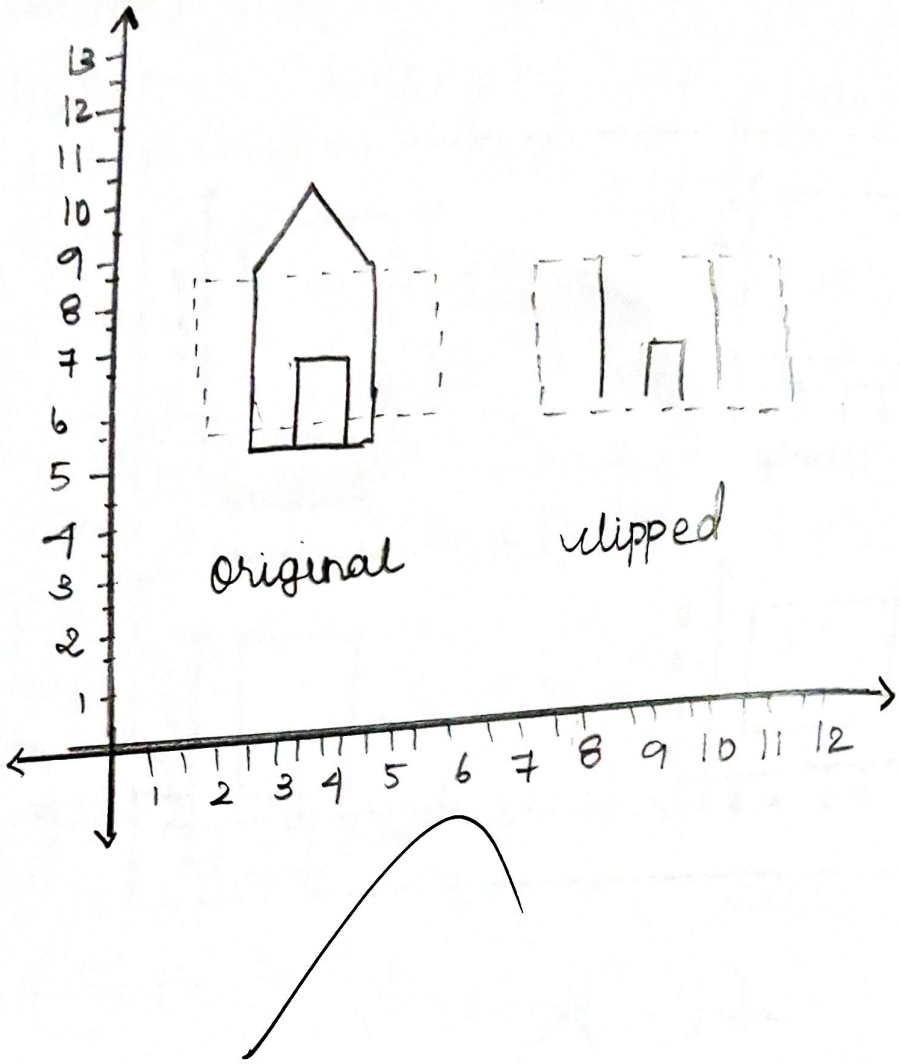
code:-

```
import numpy as np
def clip-polygon (poly, clip-win)
xmin, ymin, xmax = clip-win
edges = [
    (lambda P: P[0] >= xmin, lambda P1,
    (lambda P: P[0] <= xmax, lambda P2,
    (lambda P: P[1] >= ymin, lambda P1,
    (lambda P: P[1] >= xmin, lambda P2,
]
m = (y2-y1) / (x2-x1);
Switch edgetype
    case {1,2}
    ix = limit val;
    iy = limit val;
if clipped_x:
    clipped_x_closed = clipped_x + [clipped_x[0]]
    ax.fill (clipped_x, clipped_y, 'go'
                label = 'clipped Vertices')
ax. legend (box_to_anchor = (1.05, 1)
```

```
ax.legend(loc='upper right')
ax.plot(clipped_x, clipped_y 'go')
ax.plot(label='clipped vertices')
plt.tight_layout()
plt.show()
main()
```

Result:

Thus the code is implemented and the output is obtained successfully.


13
12
11
10
9
8
7
6
5
4
3
2
1
original
flipped
1 2 3 4 5 6 7 8 9 10 11 12

Ex 7 Create and Render 3D objects using Polygons and Apply Basic Color and Shading Techniques.

Aim:-

To Create and render 3D objects (such as cube or sphere) using polygonal modeling and apply:

→ Flat Shading.

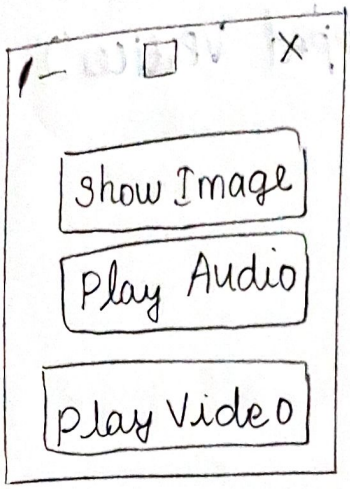
→ Basic Color techniques.

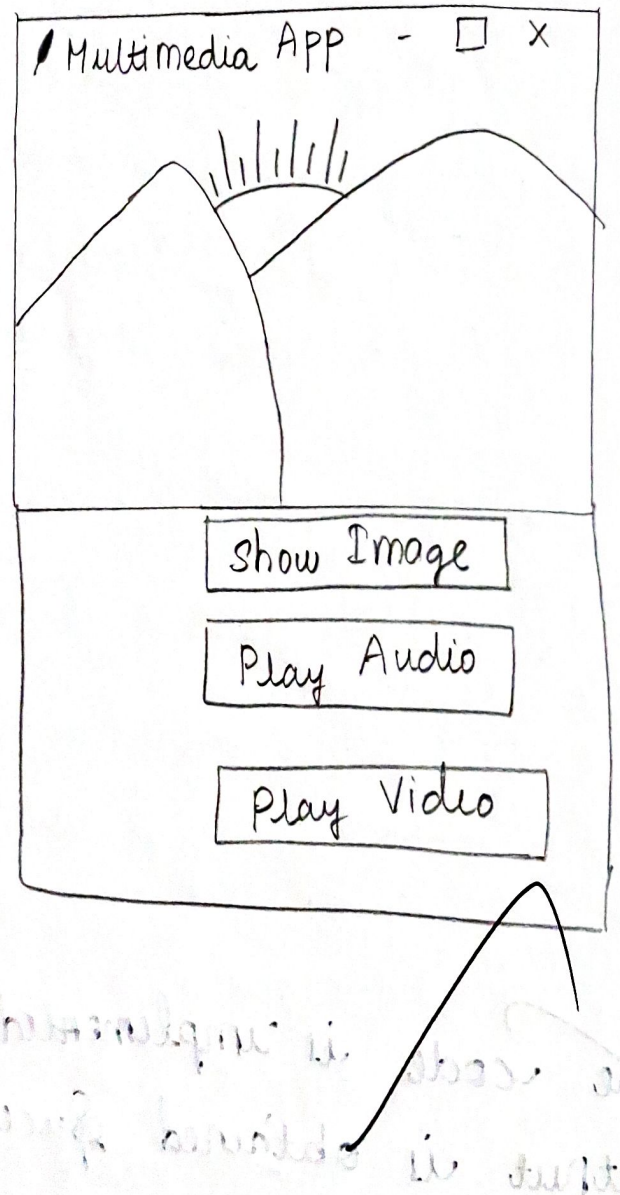
Program:

```
import matplotlib.pyplot as plt.
from mpl_toolkits.mplot3d.art3d import
        Poly3DCollection
import numpy as np
vertices = np.array([[0,0,0], [1,0,0], [1,1,0],
                     [0,1,0], [0,0,1], [1,0,1], [1,1,1]
                     [0,1,1]])
Faces = [[vertices[j] for j in [0,1,2,3]],
         [vertices[j] for j in [4,5,6,7]],
         [vertices[j] for j in [0,1,5,4]],
         [vertices[j] for j in [2,3,7,6]],
         [vertices[j] for j in [1,2,6,5]],
         [vertices[j] for j in [4,7,3,0]]]
```

Result:-

3D polygonal objects were Successfully created and rendered. Basic color and flat Shading techniques were applied to enhance visual representation.


Show Image
Play Audio
Play Video


Multimedia App
Show Image
Play Audio
Play Video

Ex:- Create a Multimedia Application that Integrates Images, Sound and Video in a Simple User Interface.

Aim:-

To develop a simple multimedia application that integrates:

- Images
- Audio
- Video

into a unified user interface using Python and suitable libraries.

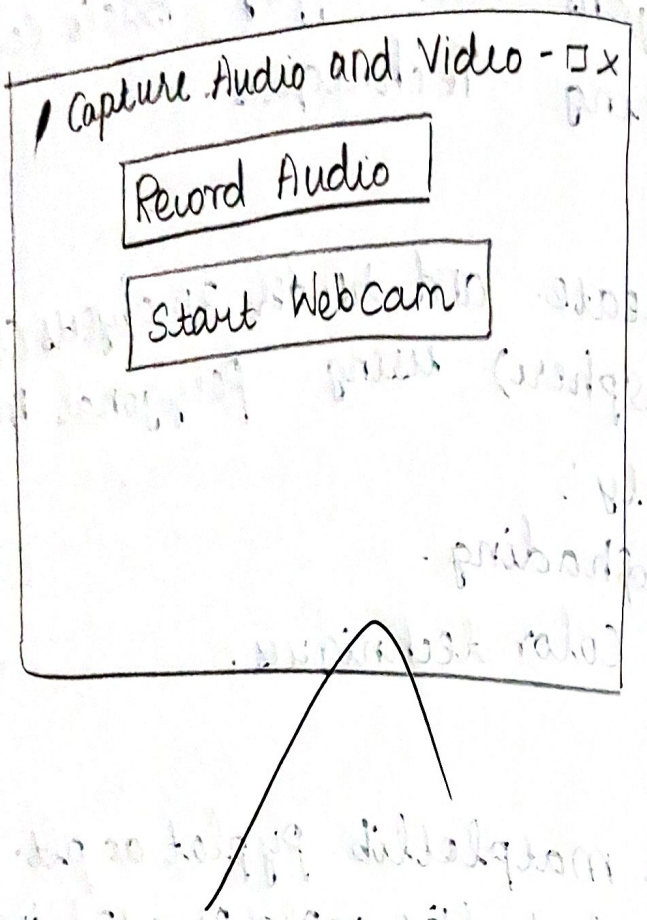
Program:-

```
import tkinter as tk
from PIL import Image, ImageTk
from playsound import playsound.
import cv2, threading.
def load_image():
    img = ImageTk.PhotoImage(Image.open
            ('sample_image.jpg').resize
                    (300, 300))
    panel.configure(image = img)
    panel.image = img .
def play_audio():
    playsound('sample_audio.mp3')
def play_video():
    cap = cv2.VideoCapture('sample_video.
                    mp4')
```

```
while cap.isOpened():
    ret, frame = cap.read()
    if not ret or cv2.waitKey(25) &
        0xFF==ord('q'): break
    cv2.imshow("Video", frame)
cap.release()
cv2.destroyAllWindows()
root = tk.Tk()
root.title("Multimedia App")
panel = tk.Label(root)
panel.pack()
tk.Button(root, text="Show Image",
          command=load_image).pack(pady=5)
tk.Button(root, text="Play Audio",
          command=lambda: threading.
          Thread(target=play_audio).start()).
          pack(pady=5)
tk.Button(root, text="Play Audio",
          command=lambda: threading.
          Thread(target=play_video).start()).
          pack(pady=5)
root.mainloop()
```

Result :-

A multimedia application was succesfully created that integrates image, audio & video funtionality into a single interactive interface.


Capture Audio and Video
Record Audio
Start Webcam

Ex:- Capture Video/Audio from Webcam or Microphone and Display on Multimedia Interface.

Aim:-

To develop a program that captures:
→ live video from the Webcam.
→ live audio from the microphone.
and displays/records them using a simple multimedia interface.

Code:-

```
import cv2, threading, tkinter as tk
import sounddevice as sd
from scipy.io.wavfile import write
def record_audio():
    print('Recording Audio...')
    fs = 44100
    audio = sd.rec(int(5 * fs), samplerate=fs,
                   channels=2)
    sd.wait()
    write('recorded_audio.wav', fs, audio)
    print('Audio saved.')
def capture_video():
    cap = cv2.VideoCapture(0)
    print('Press 'q' to stop.')
    while True:
        ret, frame = cap.read()
```

```
        if not ret or cv2.waitKey(1) & 0xFF ==
                    ord('q'): break
        cv2.imshow('Webcam Feed', frame)
    cap.release()
    cv2.destroyAllWindows()
root = tk.Tk()
root.title("Capture Audio & Video")
tk.Button(root, text="Record Audio",
    command = lambda: threading.Thread
    (target = record_audio).start()).pack(pady
                                    = 10)
tk.Button(root, text="Start Webcam",
    command = lambda: threading.Thread
    (target=capture_video).start()).pack
                                (pady=10)
root.mainloop()
```

Result:-

Live video and audio was successfully captured using webcam & microphone and displayed/saved through a multimedia interface.

Multimedia Messaging App - □ ×

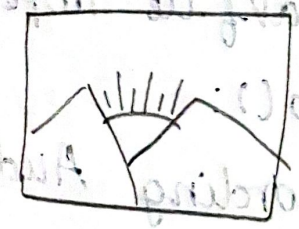
You: hello

You sent an image:

c:/users/dell/oneDrive/sample_image.jpg

You sent an audio message:

c:/users/dell/oneDrive/sample_audio.mp3



[ ] [Send Text] [Send Image] [Send Audio]

Create an Application to send and receive Multimedia Message (Text, Image, Audio).

Aim:-

To develop a basic chat-like application that allows:

→ Sending and receiving text, images and audio.

→ Using local file selection and simulated messaging UI.

Program:-

```
import tkinter as tk.
from tkinter import filedialog, scrolledtext
from PIL import Image, ImageTk
from playsound import playsound
import threading
def send_text():
    if msg := text_entry.get():
        chat_box.insert(tk.END, f"You: {msg}\n")
        text_entry.delete(0, tk.END)
def send_image():
    path = filedialog.askopenfilename(filetypes=[("Images", "*.jpeg *.png *.jpeg")])
```

chat-box.inse

Result:

A multimedia messaging interface was successfully created with options to send text, image and audio content. The application simulated messaging behaviour in a local environment.

Ex:- Create a 3D Model using Blender and Apply Basic Shading & Textures.

Aim:

To create a simple 3D model (eg. a house or object) using Blender, and apply:

→ Basic Shading.

→ Textures

Procedure:

1. Launch Blender.
2. Model Creation:
   - use add > mesh to insert basic shapes like cube, cylinder, plane etc.
   - combine the objects to form a house or a similar structure.
   - use Edit Mode for extruding and shaping.
3. Shading:
   - switch to the shading workspace.
   - Add a new material for each object.
   - Adjust Base Color, Specular, and Roughness.
4. Texturing Mapping:
   - In the Shader Editor, add an Image Texture node.
   - load an external image (eg. brick texture)

. Connect it to the Base color of the principled BSDF shadow
. Use UV Editing to align the texture.

5. lighting and Camera:
. Add light Sources.
. Position the Camera to frame the model.

6. Render Settings.
. Choose Cycle or Eevee rendering engine.
. Adjust resolution and sampling.
. Click Render > Render Image to produce the Output.

Result:-
A 3d object was Successfully modeled in blender. Basic shading and texture mapping techniques were applied, and the model was rendered using a built-in engine.